Data Engineering

Pertemuan 1

Desy Purnami Singgih Putri

Teknologi Informasi

September 15, 2026

Overview

Pertemuan 3

ETL, ELT, Data Pipeline, dan Batch Processing

From Ingested Data to Usable Data

Tujuan Pertemuan

Setelah mengikuti pertemuan ini, mahasiswa diharapkan mampu:

- menjelaskan konsep ETL dan ELT;
- membedakan pendekatan ETL dan ELT;
- memahami hubungan ingestion, transformation, storage, dan serving dalam sebuah pipeline;
- menjelaskan konsep dan karakteristik *batch processing*;
- menganalisis kebutuhan pipeline berdasarkan studi kasus;
- menentukan pendekatan ETL atau ELT berdasarkan karakteristik permasalahan; dan
- mulai merancang mini project Data Engineering.

Dari Pertemuan Sebelumnya

Pada pertemuan sebelumnya kita sudah berhasil:

mengambil data dari source system

Contoh:

- membaca CSV;
- melakukan query database;
- mengambil data melalui API;
- melakukan full dan incremental ingestion;
- melakukan validasi sederhana;
- menambahkan ingestion metadata.

Pertanyaan Berikutnya

Data sudah berhasil masuk.

Sekarang data tersebut diapakan?

Kasus Sederhana

Sebuah perusahaan memiliki data penjualan:

`sales_branch.csv`

yang berisi:

`transaction_id, customer_id, product, amount, transaction_date`

Tetapi data tersebut:

- memiliki missing value;
- format tanggal belum konsisten;
- nama produk berbeda-beda;
- terdapat duplicate transaction;
- belum memiliki informasi kategori produk.

Tujuan

Menghasilkan data yang **bersih, konsisten, dan siap dianalisis**.

Apa Itu Data Pipeline?

Data pipeline adalah rangkaian proses dan sistem yang memindahkan data melalui tahapan-tahapan Data Engineering.



Pipeline dapat sederhana:

$$\text{CSV} \rightarrow \text{Python} \rightarrow \text{SQLite}$$

atau terdiri dari banyak komponen.

Pipeline Bukan Hanya Kode

Sebuah data pipeline dapat mencakup:

- sumber data;
- proses ingestion;
- penyimpanan raw data;
- transformasi;
- penyimpanan processed data;
- scheduling;
- dependency;
- validation;
- logging;
- monitoring;
- metadata.

ETL

$$\mathbf{E} \rightarrow \mathbf{T} \rightarrow \mathbf{L}$$

Extract $\rightarrow$ Transform $\rightarrow$ Load

Ide Utama

Data **ditransformasi terlebih dahulu** sebelum dimasukkan ke destination utama.

E – Extract

Extract berarti mengambil data dari source system.

Contoh source:

- CSV / JSON / Parquet;
- database;
- REST API;
- log;
- aplikasi eksternal.

Contoh:

$$\texttt{sales_branch.csv} \rightarrow \text{Pandas DataFrame}$$

Hubungan dengan Pertemuan Sebelumnya

Extract merupakan salah satu bagian yang sudah kita praktikkan saat membahas ingestion.

T – Transform

Transform mengubah data agar sesuai dengan kebutuhan downstream.

Contoh:

- menghapus duplicate;
- menangani missing values;
- mengubah tipe data;
- standardisasi format;
- filtering;
- joining;
- aggregation;
- enrichment;
- business rules.

Contoh

"Laptop ", "laptop" dan "LAPTOP"

L – Load

Load berarti menyimpan hasil transformasi ke destination.

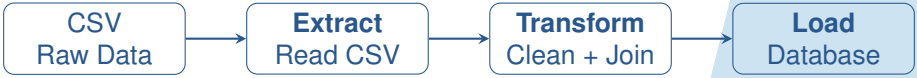
Destination dapat berupa:

- relational database;
- data warehouse;
- data lake;
- data lakehouse;
- object storage;
- analytical database.

Contoh:

Clean DataFrame → SQLite / DuckDB / Parquet

Contoh ETL



Contoh proses:

1. membaca data transaksi;
2. menghapus duplicate;
3. memperbaiki format tanggal;
4. menggabungkan master produk;
5. menghitung nilai penjualan;
6. menyimpan hasil ke analytical database.

ELT

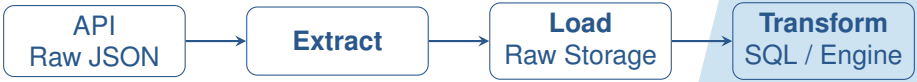
$$\mathbf{E} \rightarrow \mathbf{L} \rightarrow \mathbf{T}$$

Extract $\rightarrow$ Load $\rightarrow$ Transform

Perbedaan Utama

Data terlebih dahulu dimasukkan ke destination, kemudian transformasi dilakukan **di dalam atau dekat dengan storage tersebut**.

Contoh ELT



Raw data tetap disimpan.
Transformasi dapat dilakukan berulang kali untuk menghasilkan dataset berbeda:

$$\text{Raw Data} \rightarrow \text{Dataset A}$$
$$\text{Raw Data} \rightarrow \text{Dataset B}$$

ETL vs ELT

	ETL	**ELT**
Urutan	Extract → Transform → Load	Extract → Load → Transform
Transformasi	Sebelum data masuk destination	Setelah data masuk destination
Raw data	Belum tentu disimpan di target utama	Umumnya dapat dipertahankan
Compute	Dilakukan oleh processing layer sebelum load	Dapat memanfaatkan compute pada analytical platform
Fleksibilitas	Struktur target lebih ditentukan lebih awal	Raw data dapat digunakan kembali

Mana yang Lebih Baik?

It Depends.

Pertimbangkan:

- volume data;
- karakteristik destination;
- kemampuan compute;
- kebutuhan menyimpan raw data;
- kebutuhan downstream;
- security dan privacy;
- biaya;
- kemudahan maintenance.

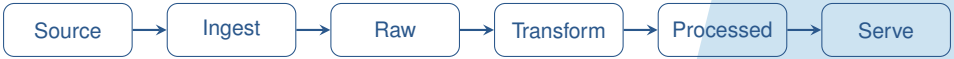
Data Engineering

Pertanyaan yang tepat bukan:

ETL/ELT Bukan Seluruh Data Pipeline

ETL atau ELT adalah salah satu pola dalam pipeline.

Pipeline lengkap dapat berupa:



Dan di seluruh proses tersebut kita juga membutuhkan:

validation – logging – metadata – monitoring – orchestration

Batch Processing

Pada **batch processing**, sekumpulan data diproses sebagai satu unit pekerjaan.

Contoh:

`sales_2026-09-14.csv`

diproses pada pukul:

23:00 setiap hari

Pipeline:

Today's Data $\rightarrow$ Process $\rightarrow$ Daily Output

Batch processing cocok ketika hasil tidak harus tersedia setiap saat.

Contoh Batch Pipeline



Contoh jadwal:

- Batch 1: 1 September;
- Batch 2: 2 September;
- Batch 3: 3 September;
- . . .

Batch Frequency

Batch dapat dijalankan:

- setiap 5 menit;
- setiap jam;
- setiap hari;
- setiap minggu;
- ketika file tersedia;
- ketika jumlah data mencapai batas tertentu.

Trade-off

Batch lebih sering:

data lebih fresh

tetapi:

compute + network + operational complexity dapat meningkat.

Dependency dalam Pipeline

Misalkan terdapat tiga proses:



Jika proses A gagal:

B seharusnya tidak berjalan.

Jika B gagal:

C seharusnya tidak menggunakan output yang salah.

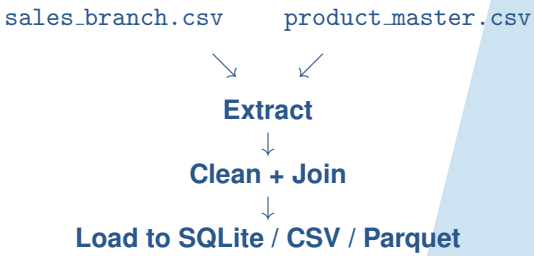
Inilah salah satu alasan pipeline perlu **workflow management**.

Demo: ETL Pipeline Sederhana

Kita akan menggunakan dua sumber:

1. `sales_branch.csv`
2. `product_master.csv`

Pipeline:



Demo – Extract

```
import pandas as pd

sales = pd.read_csv(
    "demo_data/sales_branch.csv"
)

products = pd.read_csv(
    "demo_data/product_master.csv"
)

print(sales.head())
print(products.head())
```

Extract

Pada tahap ini kita hanya **mengambil data dari source**.

Demo – Transform

```
# remove duplicate transactions
sales = sales.drop_duplicates(
    subset=["transaction_id"]
)

# standardize product name
sales["product"] = (
    sales["product"]
    .str.strip()
    .str.lower()
)

products["product"] = (
    products["product"]
    .str.strip()
    .str.lower()
)

# join with master data
result = sales.merge(
    products,
    on="product",
    how="left"
)
```

Demo – Load

Hasil transformasi kemudian disimpan.

```
import sqlite3

conn = sqlite3.connect(
    "demo_data/warehouse.db"
)

result.to_sql(
    "fact_sales",
    conn,
    if_exists="replace",
    index=False
)

conn.close()

print("ETL completed.")
```

Apakah Pipeline Sudah Selesai?

Kode berhasil dijalankan.
Tetapi Data Engineer masih perlu memeriksa:

- apakah seluruh source berhasil dibaca?
- berapa jumlah record input?
- berapa jumlah record output?
- apakah join menghasilkan missing category?
- apakah ada duplicate?
- kapan pipeline dijalankan?
- apakah proses berhasil atau gagal?

Again...

No error $\neq$ correct data pipeline.

Mulai Mencicil Mini Project

Mulai pertemuan ini, kita akan mulai menyiapkan **mini project Data Engineering**.

Project belum harus langsung menjadi pipeline lengkap.

Setiap materi berikutnya akan menambahkan satu komponen:



Prinsip

Project dikembangkan **bertahap mengikuti materi kuliah**, bukan dikerjakan sekaligus di akhir semester.

Strategi Mini Project

Terdapat **9 kelompok** dengan **9 studi kasus yang berbeda**.

Setiap kelompok menggunakan:

- data yang tersedia secara publik;
- minimal **dua sumber** / **dataset**;
- sumber berupa API, CSV, JSON, atau file publik;
- data yang dapat diakses kembali saat pipeline dijalankan.

Tidak Perlu Mencari Data Sendiri dari Nol

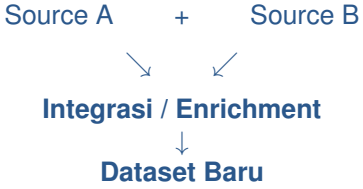
Sumber data utama sudah ditentukan.
Kelompok tetap dapat menentukan **scope, kebutuhan, dan output dataset** sesuai tema project.

Mengapa Minimal Dua Sumber Data?

Tujuan mini project bukan hanya:

`download data` → `save data`

Tetapi mahasiswa harus mengalami proses:



Integrasi dapat berupa:

- join berdasarkan ID;
- join berdasarkan negara dan tahun;
- enrichment metadata;
- lookup;

Mini Project: Kelompok 1–3

Kel.	Tema	Source 1	Source 2
1	**Weather Data Pipeline**	Open-Meteo Weather API (JSON)	GeoNames Cities / Indonesia (downloadable data)
2	**Airport Infrastructure Pipeline**	OurAirports `airports.csv`	OurAirports `runways.csv`
3	**Global Development Pipeline**	World Bank Population Indicator	World Bank GDP Indicator

Target

Masing-masing kelompok menghasilkan **dataset terintegrasi baru**, bukan sekadar menyalin source.

Mini Project: Kelompok 4–6

Kel.	Tema	Source 1	Source 2
4	**Energy & Emission Pipeline**	Our World in Data Energy Dataset	Our World in Data CO_2 Dataset
5	**Scientific Publication Pipeline**	Crossref REST API (publication metadata)	DOAJ Journal Dataset (journal metadata)
6	**Digital Books Pipeline**	Project Gutenberg Catalog	Gutendex API (book metadata)

Mini Project: Kelompok 7–9

Kel.	Tema	Source 1	Source 2
7	**Food Nutrition Pipeline**	USDA FoodData Central `food.csv`	USDA FoodData Central `food_nutrient.csv`
8	**Natural Events Pipeline**	NASA EONET Events API	NASA EONET Categories API
9	**Open Source Repository Pipeline**	GitHub Repository API	GitHub Issues / Commits API

Contoh Integrasi: Airport Pipeline

Kelompok 2 memperoleh:

airports.csv

- airport ID
- airport name
- country
- latitude
- longitude
- airport type

runways.csv

- airport reference
- runway length
- runway width
- surface
- closed / open

Kemudian:

$$\texttt{airports} \bowtie \texttt{runways}$$

menggunakan:

Contoh Integrasi: World Bank

Kelompok 3 memiliki dua indikator:

Population + **GDP**

Masing-masing memiliki informasi:

`country` `year` `value`

Integrasi dilakukan berdasarkan:

`country + year`

Hasil:

`country | year | population | GDP`

Contoh Integrasi: Publication Pipeline

Kelompok 5 menggunakan:

Crossref

Metadata artikel:

- title;
- DOI;
- author;
- publication year;
- ISSN.

DOAJ

Metadata jurnal:

- journal title;

Minimum Requirement Semua Kelompok

Pada akhir semester, pipeline minimal memiliki:

1. minimal **dua source**;
2. ingestion yang dapat dijalankan ulang;
3. penyimpanan **raw data**;
4. proses cleaning / transformation;
5. integrasi data;
6. penyimpanan **processed data**;
7. workflow atau automation;
8. data quality checks;
9. logging / monitoring sederhana;
10. metadata dan dokumentasi.

Tidak Wajib

Tidak wajib menggunakan Kafka, Spark, Hadoop, Kubernetes, atau cloud berbayar.

Teknologi yang Disarankan

Agar semua project memiliki tingkat kompleksitas yang relatif seimbang:

- **Python**;
- **Pandas**;
- **Requests**;
- CSV / JSON / Parquet;
- SQLite atau DuckDB;
- SQL;
- library tambahan bila memang diperlukan.

Prinsip

Gunakan teknologi secukupnya.

Pipeline yang sederhana tetapi reliable lebih baik daripada arsitektur kompleks yang tidak dipahami.

Sprint 0: Tugas Minggu Ini

Belum perlu membangun pipeline lengkap.

Setiap kelompok melakukan:

1. pelajari kedua source data;
2. download / akses sample data;
3. tampilkan minimal 5 record dari masing-masing source;
4. identifikasi struktur / schema data;
5. tentukan atribut yang akan digunakan;
6. tentukan bagaimana kedua data dapat diintegrasikan;
7. tentukan calon output dataset;
8. tentukan pendekatan awal ETL atau ELT;
9. gambar arsitektur awal pipeline.

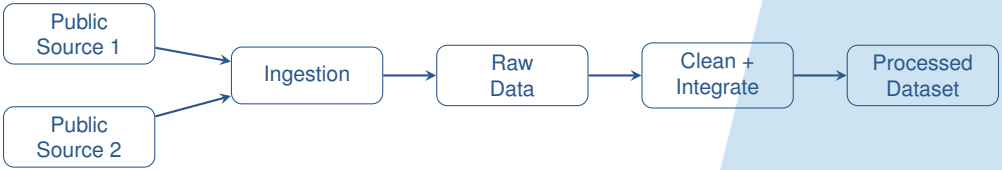
Deliverable Sprint 0

Setiap kelompok mengumpulkan dokumen singkat yang berisi:

1. **Judul Project**
2. **Use Case**
3. **Target User**
4. **Source 1**
5. **Source 2**
6. contoh 5 record masing-masing source
7. daftar atribut yang akan digunakan
8. **integration key** / **integration logic**
9. rancangan output dataset
10. pilihan awal ETL atau ELT beserta alasan
11. diagram arsitektur awal

Belum Perlu

Contoh Arsitektur Sprint 0



Pada Sprint 0 yang terpenting adalah:

Source apa? Hubungannya apa? Output yang ingin dihasilkan apa?

Yang Akan Dinilai

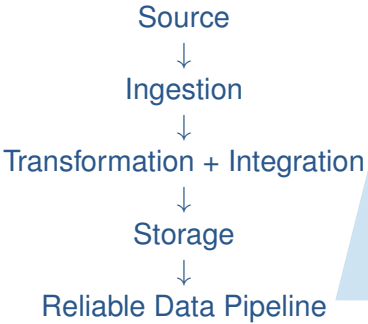
Aspek	Bobot
Analisis kebutuhan dan arsitektur pipeline	20%
Akuisisi, transformasi, integrasi, dan storage	30%
Workflow, automation, dan integrasi pipeline	20%
Data quality, reliability, dan dokumentasi	20%
Presentasi dan demonstrasi	10%
Total	**100%**

Yang Tidak Dinilai

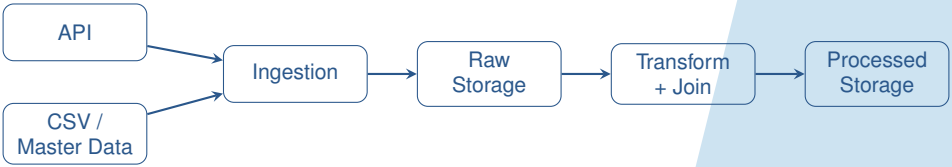
Banyaknya teknologi yang digunakan **bukan indikator kualitas project**.

Yang Perlu Diingat

Project berbeda,
learning objective tetap sama.



Arsitektur Minimum Project



Nanti secara bertahap kita tambahkan:

Orchestration + Quality + Logging + Monitoring + Metadata

Raw dan Processed Layer

Struktur sederhana yang dapat digunakan semua kelompok:

```
project/
 |-- raw/
|-- processed/
 |-- scripts/
  |-- logs/
 |-- config/
|-- README.md
```

Contoh:

API JSON $\rightarrow$ `raw/weather_20260914.json`

Transform

$\downarrow$

`processed/weather.parquet`

Sprint 0 – Mulai Minggu Ini

Belum perlu membuat pipeline lengkap.

Setiap kelompok mulai dengan:

1. memahami tema project;
2. menentukan **siapa pengguna datanya**;
3. menentukan pertanyaan / kebutuhan data;
4. mempelajari API yang diberikan;
5. berhasil mengambil **sample data**;
6. menentukan source data kedua;
7. menggambar arsitektur awal;
8. menentukan apakah lebih cocok ETL atau ELT.

Target

Kita mulai dari desain kecil, lalu pipeline akan berkembang mengikuti materi kuliah.

Deliverable Sprint 0

Setiap kelompok mengumpulkan **1 halaman** yang berisi:

1. **Judul Project**
2. **Use Case**
3. **Target User**
4. **Source 1**: API
5. **Source 2**: CSV / file / master data
6. contoh data dari masing-masing source
7. pilihan awal: ETL atau ELT
8. diagram arsitektur awal

Belum Dinilai dari Kompleksitas

Yang dilihat adalah:

Apakah kebutuhan dan aliran datanya masuk akal?

Pertanyaan yang Harus Dijawab Kelompok

Sebelum mulai coding:

1. Data ini digunakan untuk apa?
2. Siapa pengguna akhirnya?
3. Data apa yang benar-benar dibutuhkan?
4. Seberapa sering data harus diambil?
5. Apakah full atau incremental ingestion?
6. Data apa yang perlu dibersihkan?
7. Apa yang perlu digabungkan?
8. Di mana raw data disimpan?
9. Di mana processed data disimpan?
10. Bagaimana kita tahu pipeline berhasil?

Perkembangan Project Selama Semester

Tahap	Materi Kuliah	Yang Ditambahkan ke Project
Sekarang	ETL / ELT	Use case, source, architecture, ETL/ELT decision
Berikutnya	Transformation	Cleaning, standardization, join, validation
Storage	Warehouse / Lake	Raw dan processed storage
Checkpoint	Project Design	Finalisasi rancangan pipeline
Orchestration	Workflow	Scheduling, dependency, retry
Data Quality	Testing	Quality rules dan validation
Metadata	Governance	Metadata, lineage, documentation
Finalisasi	Reliability	Testing, logging, monitoring, deployment

Penilaian Mini Project

Penilaian akhir project mencakup:

Aspek	Bobot
Analisis kebutuhan dan arsitektur pipeline	20%
Akuisisi, transformasi, integrasi, dan storage	30%
Workflow, automation, dan integrasi pipeline	20%
Data quality, reliability, dan dokumentasi	20%
Presentasi dan demonstrasi	10%
Total	**100%**

Yang Tidak Menjadi Fokus Project

Project ini **bukan** lomba:

- siapa yang membuat dashboard paling cantik;
- siapa yang memakai teknologi paling banyak;
- siapa yang menggunakan cloud paling mahal;
- siapa yang memiliki kode paling panjang.

Dashboard boleh dibuat sebagai tambahan, tetapi bukan tujuan utama.

Fokus

Reliable Data Flow

dari:

Source → Usable Data

Project Tidak Harus “Big Data”

Data tidak harus berukuran terabyte.

Dataset kecil tetap dapat digunakan untuk belajar:

- ingestion;
- ETL / ELT;
- pipeline design;
- automation;
- validation;
- metadata;
- reliability;
- reproducibility.

Lebih Penting

Pipeline yang kecil tetapi **jelas, benar, repeatable, dan dapat dijelaskan**

Studi Kasus

Sebuah perusahaan memiliki:

- transaksi dari database;
- master produk dari CSV;
- data harga kompetitor dari API.

Laporan hanya diperlukan setiap pagi.

Diskusikan:

1. Apakah batch atau streaming?
2. Full atau incremental ingestion?
3. ETL atau ELT?
4. Apa raw data yang perlu disimpan?
5. Transformasi apa yang dibutuhkan?
6. Apa destination akhirnya?

Tidak Ada Satu Jawaban

Dua kelompok dapat menghasilkan arsitektur berbeda untuk kasus yang sama.

Keduanya dapat benar jika:

- memenuhi kebutuhan pengguna;
- trade-off dapat dijelaskan;
- data tidak hilang;
- pipeline dapat dijalankan kembali;
- hasil dapat divalidasi;
- kompleksitas masih masuk akal.

Design decisions need reasons.

Takeaways

1. Data pipeline adalah rangkaian sistem dan proses yang membawa data melalui Data Engineering Lifecycle.
2. ETL menggunakan pola Extract–Transform–Load.
3. ELT menggunakan pola Extract–Load–Transform.
4. Tidak ada pendekatan yang selalu lebih baik; pemilihan bergantung kebutuhan.
5. Batch processing memproses data dalam kelompok pada interval tertentu.
6. Project semester akan dikembangkan sedikit demi sedikit mengikuti materi perkuliahan.
7. Semua kelompok menggunakan kerangka teknis yang sebanding, tetapi memiliki domain dan data yang berbeda.

Pertemuan Berikutnya

Sekarang pipeline kita sudah:

$$\text{Source} \rightarrow \text{Ingest} \rightarrow \textbf{Transform} \rightarrow \text{Store}$$

Pada pertemuan berikutnya kita akan membahas transformasi lebih dalam:

- cleaning;
- filtering;
- standardization;
- missing values;
- deduplication;
- joining;
- schema validation;
- menyimpan hasil transformasi.

Raw Data → Trusted Data

The End